

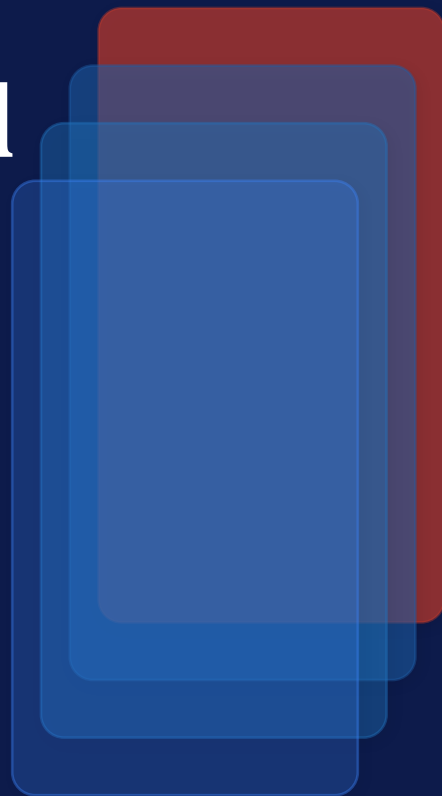
Municipalidad Valle del Sol

Plataforma de Gestión de Emergencias

Arquitectura de Microservicios · Spring Boot · Next.js · Kubernetes

Integrantes: Lucas Soto
Joaquín Beltrán

Fecha: Junio 2026



El Problema

Sin digitalización

Reportes de incendios por llamada o redes sociales. Sin trazabilidad ni estructura.

Sin visualización

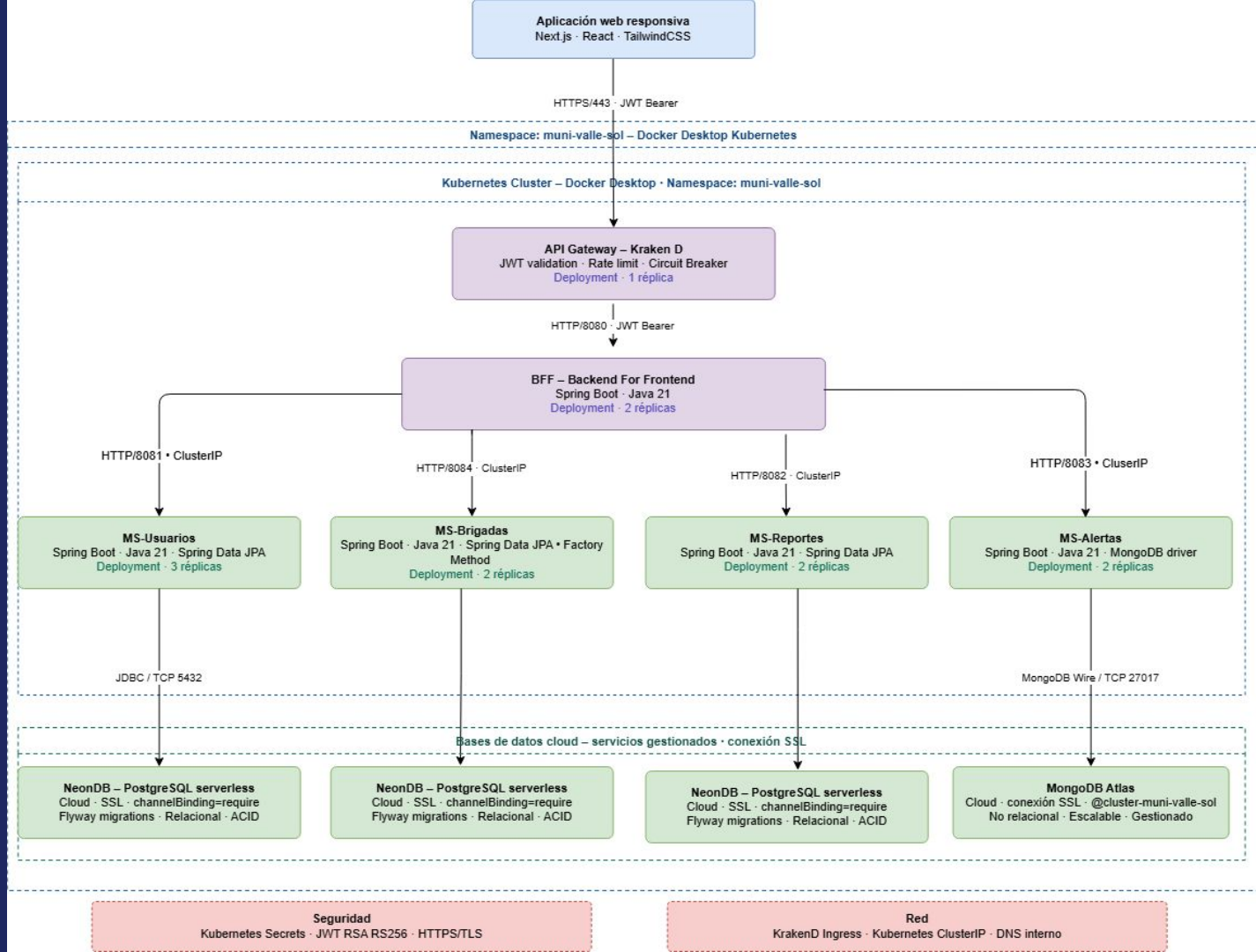
Imposible ver la ubicación de focos en tiempo real ni coordinar unidades en campo.

Sin integración

Bomberos, brigadistas y municipalidad operaban en silos sin datos compartidos.

Solución: Plataforma digital unificada con arquitectura de microservicios

Arquitectura General



Los 4 Microservicios

MS-Usuarios

:8081

PostgreSQL Neon

Spring Security · JWT RS256 · Flyway

Autenticación, roles ADMIN / FUNCIONARIO / CIUDADANO

MS-Alertas

:8083

MongoDB Atlas

Spring Data MongoDB · Factory Method

Alertas comunitarias con severidad HIGH/MEDIUM/LOW

MS-Reportes

:8082

PostgreSQL Neon

JPA · Flyway · Factory Method

Registro de incendios con GPS, tipos y prioridades automáticas

MS-Brigadas

:8084

PostgreSQL Neon

JPA · Flyway · Factory Method

Gestión de brigadas INCENDIO/RESCATE/ubicaciones

Stack Tecnológico

Capa	Tecnología
Frontend	Next.js 16 · TypeScript · React 19 · MapLibre GL
API Gateway	KrakenD · Go
BFF	Spring Boot · Java 21 · Resilience4j
Microservicios	Spring Boot · JPA · Flyway · Swagger
Base de datos	Neon DB PostgreSQL × 3 · MongoDB Atlas × 1
Testing	JUnit 5 · Mockito · JaCoCo · Vitest
Infraestructura	Docker · Docker Compose · Kubernetes

BFF · Seguridad & RBAC

Flujo de Autenticación JWT RS256

1. POST /auth/login → KrakenD → BFF
2. BFF → MS-Usuarios: valida BCrypt + genera JWT firmado con RSA private_key
3. BFF setea cookie HttpOnly: access_token=JWT (no accesible desde JS)
4. Requests siguientes envían cookie automáticamente
5. JwtAuthFilter valida firma RSA con clave pública cacheada desde JWKS
6. Claims: sub (email), rol, exp → aplica RBAC

Control de Acceso (RBAC)

Ruta	Rol requerido
GET /alertas/**	Cualquiera
POST /reportes	Cualquiera
POST/PUT/DELETE /alertas	ADMIN / FUNC.
/brigadas/**	ADMIN / FUNC.
GET/PUT/DELETE /reportes	ADMIN

Circuit Breaker (Resilience4j): Si MS-Usuarios o MS-Reportes fallan respuesta de contingencia sin colapsar la plataforma.

Persistencia de Datos

Principio: Database per Service — cada MS tiene su propia BD aislada en la nube

PG

MS-Usuarios

NeonDB PostgreSQL

usuario · rol (ENUM)
Flyway V2 · ddl-auto: validate
BCrypt passwords

PG

MS-Reportes

NeonDB PostgreSQL

reporte · tipo (ENUM)
latitud / longitud (GPS)
Flyway V1 · Factory states

MDB

MS-Alertas

MongoDB Atlas

collection: alerts
severity (HIGH/MED/LOW)
sin StatefulSets en K8s

PG

MS-Brigadas

NeonDB PostgreSQL

brigada · tipo · estado
latitud / longitud
Flyway V1 · 4 estados

Patrones de Diseño Aplicados

Factory Method

MS-Reportes · MS-Alertas · MS-Brigadas

Crea entidades con reglas de negocio por tipo.
INCENDIO → PRIORIDAD ALTA automático.

Circuit Breaker

BFF → Resilience4j

MS caído no colapsa la plataforma. Fallback con respuesta de contingencia.

Repository

Todos los MS

Spring Data JPA/MongoDB. Desacopla lógica de negocio del acceso a datos.

Gateway / BFF

KrakenD + BFF

Punto único de entrada. BFF oculta la complejidad de 4 MS al frontend (Facade).

JWKS Caching

BFF → JwtService

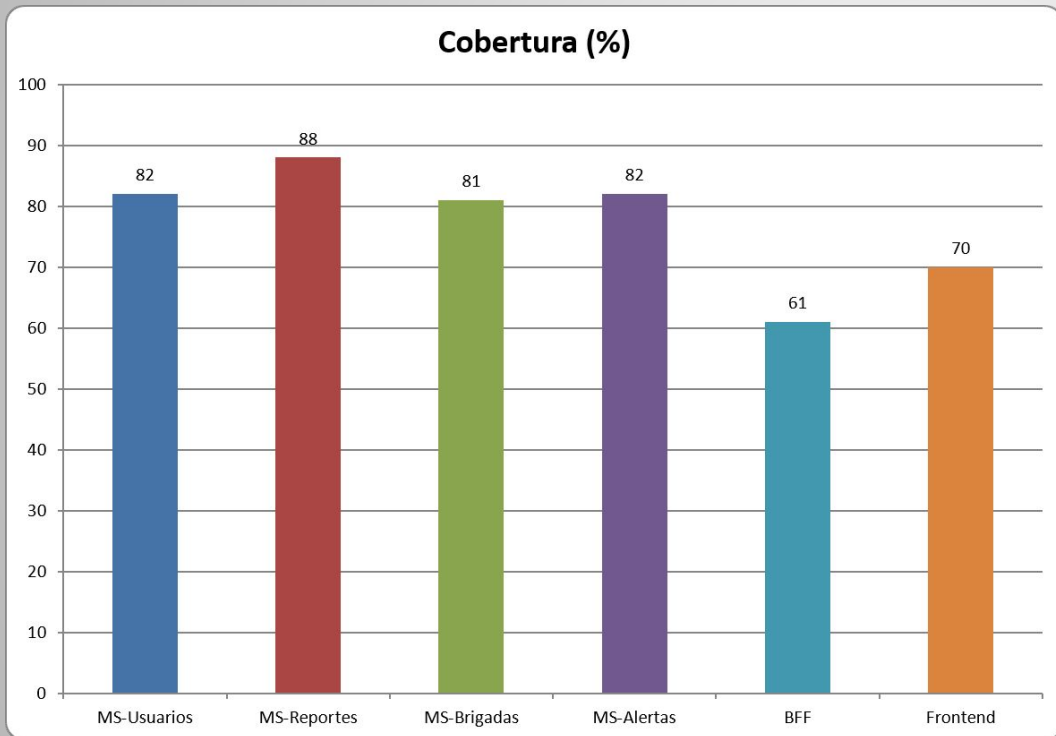
Clave pública RSA cacheada. Refresco cada 30 min. Soporta rotación de claves.

Singleton (Zustand)

Frontend

useAuthStore · useAlertStore · useAppStore:
estado global único en el cliente.

Pruebas Unitarias - Cobertura > 60%



60% mínimo

Herramientas

JUnit 5 + Mockito

Backend (todos los MS + BFF)

Spring Boot Test

MockMvc, WebTestClient

JaCoCo

Reporte HTML en target/site/jacoco/

Vitest + Testing Library

Frontend Next.js (jsdom)

```
./mvnw test jacoco:report | npm run test:coverage
```

Demo · Flujos Principales

1. Login de Ciudadano

1

Browser → POST /api/auth/login → KrakenD

2

BFF valida BCrypt en MS-Usuarios → genera JWT RS256

3

Set-Cookie HttpOnly: access_token=JWT

4

AuthStore actualiza estado → redirect a Dashboard

2. Ciudadano Crea Reporte

1

POST /api/reportes → cualquier rol autenticado

2

ReportFactory: INCENDIO → estado=ACTIVO + [PRIORIDAD ALTA]

3

Guardado en PostgreSQL Neon con GPS (lat/Ing)

4

MapLibre GL actualiza mapa con nuevo marcador

3. Admin Emite Alerta

1

POST /reportes/{id}/emitir-alerta → requiere ADMIN

2

AlertFactory: status=ACTIVE, date=now(), severity=HIGH

3

Guardado en MongoDB Atlas

4

AlertPanel del frontend recibe alerta nueva en polling 30s

Despliegue · Docker & Kubernetes

Recurso K8s	Componentes	Réplicas
Deployment	KrakenD, BFF, 4 MS, Frontend	BFF: 2 · FE: 2
Service ClusterIP	Todos los MS y BFF	DNS interno
Service LoadBalancer	Frontend Next.js	Externo
ConfigMap	KrakenD (krakend.json)	—
Secret (muni-secrets)	Credenciales BD + JWT	—
Secret (muni-jwt-keys)	RSA private + public key	—
Namespace	muni-valle-sol	Aislado

Docker Multi-Stage Build

Etapas 1: JDK 21 Alpine (compilación)

Etapas 2: JRE 21 Alpine (runtime)

Imagen final ~60% más liviana

Node 20 Alpine para Next.js

Docker Compose (Dev Local)

```
docker compose up --build
# Levanta los 5 servicios
# backend en muni-network
```

Conclusiones

Arquitectura robusta

4 MS independientes + BFF + KrakenD + Next.js en Kubernetes con namespace dedicado.

Calidad validada

Cobertura >60% en todos los componentes. JUnit 5 + Mockito + Vitest + JaCoCo.

Resiliente y escalable

Circuit Breaker Resilience4j · K8s réplicas automáticas · multi-stage Docker builds.

Persistencia garantizada

Database per Service: NeonDB serverless × 3 + MongoDB Atlas × 1. Flyway migrations.